WEEK 6 – PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM

1. Scenario

The Art Work Shop database includes a Product Review and Rating Management System to collect customer feedback about products. Customers can provide ratings and comments to express their satisfaction. This module connects the existing Customer and Product tables and helps analyze product performance using review counts and average ratings.

2. Create Product Review Table

The Product_Review table stores customer reviews and ratings. Review_ID is the primary key, while Customer_ID and Product_ID connect the table with the existing Customer and Product tables.

```
SQL> CREATE TABLE Product_Review (
  2      Review_ID INT PRIMARY KEY,
  3      Customer_ID VARCHAR2(20) NOT NULL,
  4      Product_ID VARCHAR2(7) NOT NULL,
  5      Rating INT NOT NULL,
  6      Review_Comment VARCHAR2(500),
  7      Review_Date DATE DEFAULT SYSDATE,
  8      CONSTRAINT CHK_REVIEW_RATING CHECK (Rating BETWEEN 1 AND 5),
  9      CONSTRAINT FK_REVIEW_CUSTOMER FOREIGN KEY (Customer_ID)
 10          REFERENCES Customer(Customer_ID),
 11      CONSTRAINT FK_REVIEW_PRODUCT FOREIGN KEY (Product_ID)
 12          REFERENCES Product(Product_ID)
 13  );

Table created.

SQL> DESC Product_Review;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------
----
 REVIEW_ID                                 NOT NULL NUMBER(38)
 CUSTOMER_ID                               NOT NULL VARCHAR2(20)
 PRODUCT_ID                                NOT NULL VARCHAR2(7)
 RATING                                    NOT NULL NUMBER(38)
 REVIEW_COMMENT                                     VARCHAR2(500)
 REVIEW_DATE                                        DATE
```

3. Apply Constraints and Establish Relationships

Primary Key: Review_ID uniquely identifies each review.

Foreign Keys: Customer_ID references the Customer table, and Product_ID references the Product table.

NOT NULL: Customer ID, Product ID and Rating must be provided.

CHECK Constraint: The rating must be between 1 and 5 stars.

DEFAULT: Review_Date automatically stores the current date when a review is inserted without a specified date.

The Product Review module is connected to the existing Customer and Product modules through foreign key relationships.

4. Insert Values into Product_Review Table

The following sample records were inserted to demonstrate customer feedback and product ratings.

```
SQL> INSERT INTO Product_Review
  2  (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment)
  3  VALUES
  4  (1, 'C-10001', 'P100001', 5, 'Excellent product');

1 row created.

SQL> INSERT INTO Product_Review
  2  (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment)
  3  VALUES
  4  (2, 'C-10001', 'P100002', 4, 'Good quality product');

1 row created.

SQL> INSERT INTO Product_Review
  2  (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment)
  3  VALUES
  4  (3, 'C-10001', 'P100003', 3, 'Average product');

1 row created.

SQL> INSERT INTO Product_Review
  2  (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment)
  3  VALUES
  4  (4, 'C-10001', 'P100004', 5, 'Excellent quality');

1 row created.

SQL> INSERT INTO Product_Review
  2  (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment)
  3  VALUES
  4  (5, 'C-10001', 'P100005', 2, 'Needs improvement');

1 row created.

SQL> COMMIT;
```

Review ID 5 was later deleted as part of the DELETE operation. Therefore, four reviews remain in the final table.

5. Update and Delete Operations

Update a Review Comment

```
SQL> UPDATE Product_Review
  2  SET Review_Comment = 'Excellent quality and value'
  3  WHERE Review_ID = 1;

1 row updated.
```

This query updates the comment for Review ID 1.

Delete a Review Record

```
SQL> DELETE FROM Product_Review
  2  WHERE Review_ID = 5;

1 row deleted.
```

This query removes the review with Review ID 5.

6. Retrieve Product Review Details

Display All Reviews for a Product

```
SQL> SELECT *
  2  FROM Product_Review
  3  WHERE Product_ID = 'P100001';

 REVIEW_ID CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------
         1 C-10001              P100001          5
Excellent quality and value
16-OCT-26
```

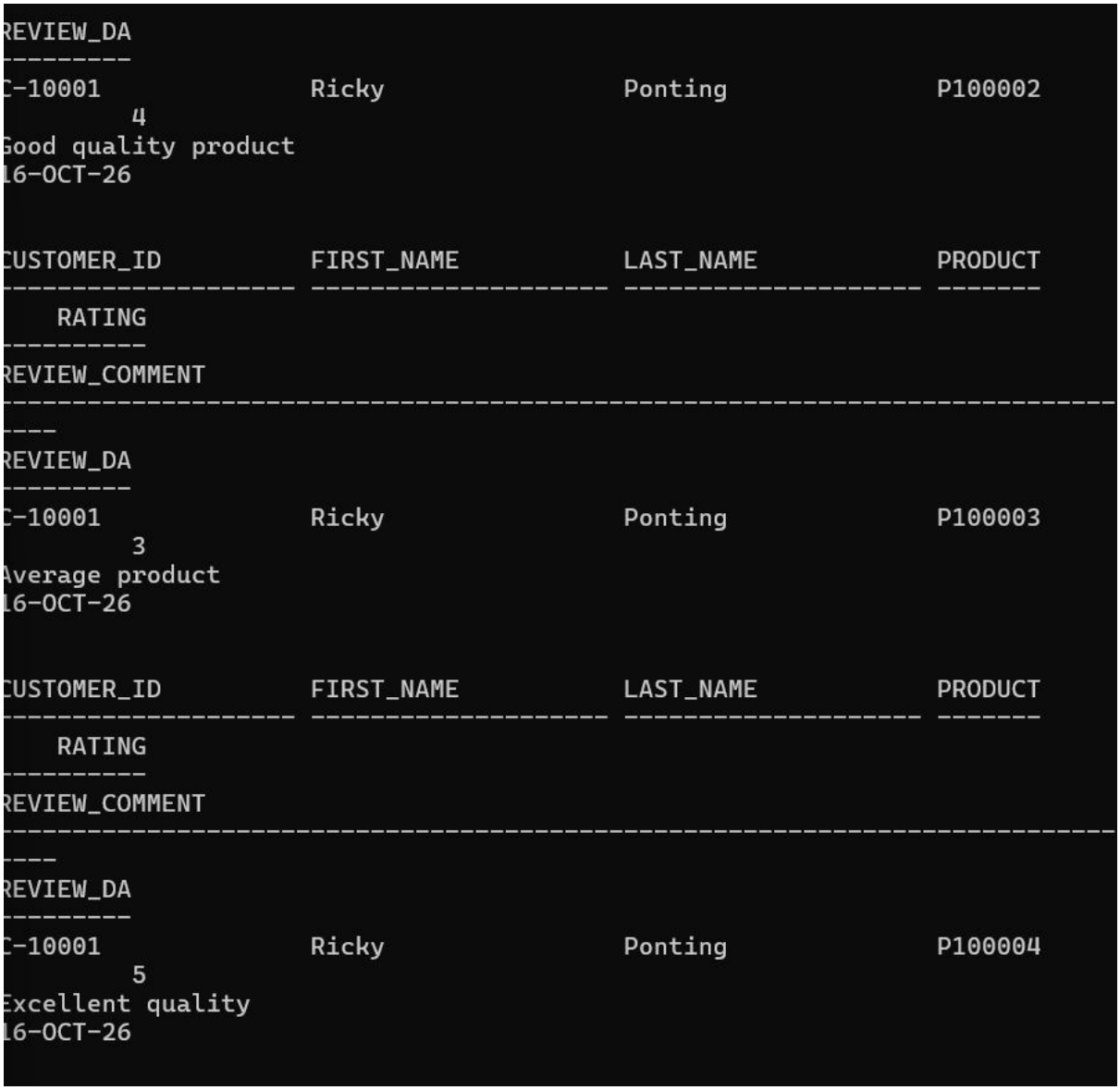
This query displays all reviews associated with Product ID P100001.

Display Customer Details with Reviews

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name,
  3         C.Last_Name,
  4         R.Product_ID,
  5         R.Rating,
  6         R.Review_Comment,
  7         R.Review_Date
  8  FROM Customer C
  9  JOIN Product_Review R
 10  ON C.Customer_ID = R.Customer_ID;

CUSTOMER_ID          FIRST_NAME           LAST_NAME            PRODUCT
-------------------- -------------------- -------------------- -------
    RATING
----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------
C-10001              Ricky                Ponting              P100001
         5
Excellent quality and value
16-OCT-26


CUSTOMER_ID          FIRST_NAME           LAST_NAME            PRODUCT
-------------------- -------------------- -------------------- -------
    RATING
----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------
C-10001              Ricky                Ponting              P100002
         4
Good quality product
16-OCT-26
```



This query displays customer details along with their product reviews and ratings.

Find Products Having Maximum Reviews

```
SQL> SELECT Product_ID, COUNT(*) AS Total_Reviews
  2  FROM Product_Review
  3  GROUP BY Product_ID
  4  ORDER BY Total_Reviews DESC;

PRODUCT TOTAL_REVIEWS
------- -------------
P100001             1
P100004             1
P100002             1
P100003             1
```

This query counts reviews for each product and displays products with the highest review counts first.

Display Recent Customer Feedback

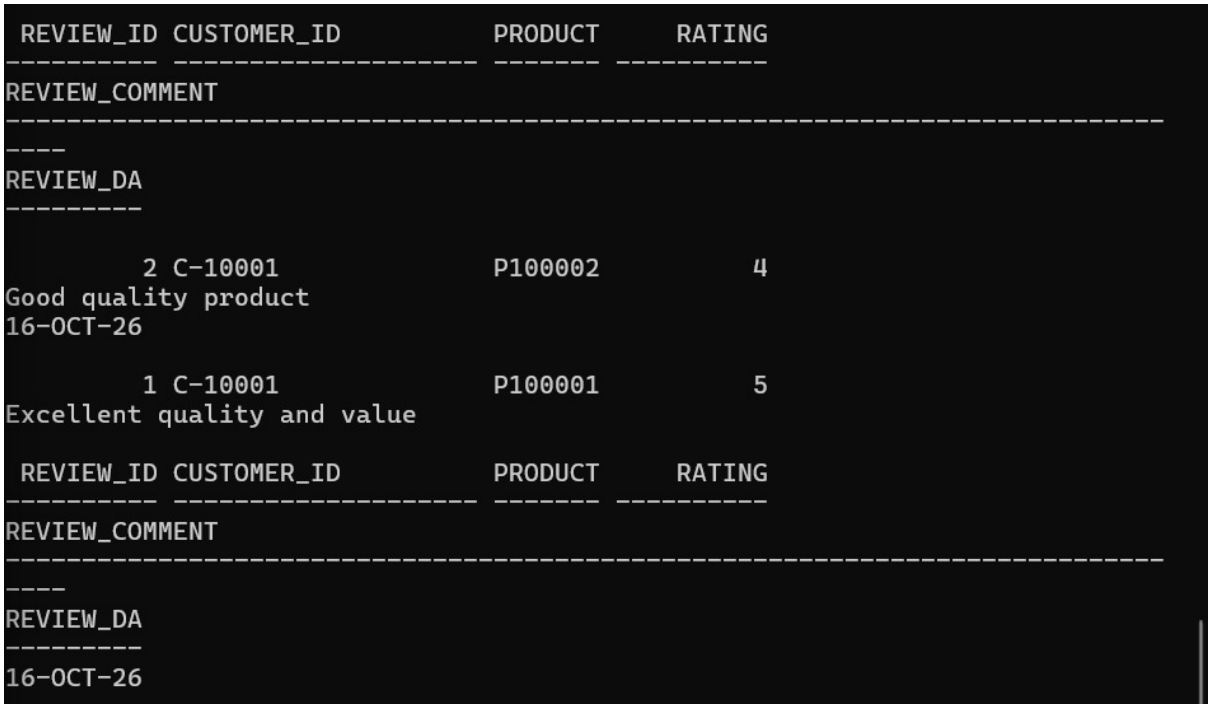
```
SQL> SELECT Product_ID, COUNT(*) AS Total_Reviews
  2  FROM Product_Review
  3  GROUP BY Product_ID
  4  ORDER BY Total_Reviews DESC;

PRODUCT TOTAL_REVIEWS
------- -------------
P100001             1
P100004             1
P100002             1
P100003             1

SQL> SELECT *
  2  FROM Product_Review
  3  ORDER BY Review_Date DESC;

 REVIEW_ID CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------
         4 C-10001              P100004          5
Excellent quality
16-OCT-26

         3 C-10001              P100003          3
Average product
16-OCT-26
```



```
 REVIEW_ID CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------

         2 C-10001              P100002          4
Good quality product
16-OCT-26

         1 C-10001              P100001          5
Excellent quality and value

 REVIEW_ID CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------
16-OCT-26
```

This query displays the most recent reviews first.

Retrieve Reviews with Ratings Above 4

```
SQL> SELECT *
  2  FROM Product_Review
  3  ORDER BY Review_Date DESC;

 REVIEW_ID CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------
         4 C-10001              P100004          5
Excellent quality
16-OCT-26

         3 C-10001              P100003          3
Average product
16-OCT-26

 REVIEW_ID CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------

         2 C-10001              P100002          4
Good quality product
16-OCT-26

         1 C-10001              P100001          5
Excellent quality and value
```

```
 REVIEW_ID CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------------
----
REVIEW_DA
---------
16-OCT-26
```

7. Calculate Average Product Ratings

Calculate Average Rating for Each Product

```
SQL> SELECT Product_ID,
  2         AVG(Rating) AS Average_Rating
  3  FROM Product_Review
  4  GROUP BY Product_ID;

PRODUCT AVERAGE_RATING
------- --------------
P100001              5
P100003              3
P100002              4
P100004              5
```

This query calculates the average rating for each product.

Count the Number of Reviews per Product

```
SQL> SELECT Product_ID,
  2         COUNT(*) AS Total_Reviews
  3  FROM Product_Review
  4  GROUP BY Product_ID;

PRODUCT TOTAL_REVIEWS
------- -------------
P100001             1
P100003             1
P100002             1
P100004             1
```

This query counts the total reviews received by each product.

Find Highest-Rated Products

```
SQL> SELECT Product_ID,
  2         AVG(Rating) AS Average_Rating
  3  FROM Product_Review
  4  GROUP BY Product_ID
  5  ORDER BY Average_Rating DESC;

PRODUCT AVERAGE_RATING
------- --------------
P100001              5
P100004              5
P100002              4
P100003              3
```

Identify Products with Average Rating Above 4

```
SQL> SELECT Product_ID,
  2         AVG(Rating) AS Average_Rating
  3  FROM Product_Review
  4  GROUP BY Product_ID
  5  HAVING AVG(Rating) > 4;

PRODUCT AVERAGE_RATING
------- --------------
P100001              5
P100004              5
```

8. Product Rating and Customer Feedback Reports

Report 1: Product Rating Analysis

This report displays the product name, number of reviews and average rating for each product.

```
SQL> SELECT P.Product_ID,
  2         P.Product_Name,
  3         COUNT(R.Review_ID) AS Total_Reviews,
  4         AVG(R.Rating) AS Average_Rating
  5  FROM Product P
  6  LEFT JOIN Product_Review R
  7  ON P.Product_ID = R.Product_ID
  8  GROUP BY P.Product_ID, P.Product_Name;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
----
TOTAL_REVIEWS AVERAGE_RATING
------------- --------------
P100004
Gaming Controller
            1              5

P100007
Gaming Monitor
            0

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
----
TOTAL_REVIEWS AVERAGE_RATING
------------- --------------

P100003
Mechanical Keyboard
            1              3

P100008
Power Bank

PRODUCT
```

```
----
TOTAL_REVIEWS AVERAGE_RATING
------------- --------------
            0

P100002
Wireless Mouse
            1              4

P100005

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
----
TOTAL_REVIEWS AVERAGE_RATING
------------- --------------
Smartphone
            0

P100001
Gaming Headset
            1              5


PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
----
TOTAL_REVIEWS AVERAGE_RATING
------------- --------------
P100006
Bluetooth Speaker
            0


8 rows selected.
```

Report 2: Customer Feedback Analysis

Most Reviewed Products

```
SQL> SELECT P.Product_ID,
  2         P.Product_Name,
  3         COUNT(R.Review_ID) AS Total_Reviews
  4  FROM Product P
  5  LEFT JOIN Product_Review R
  6  ON P.Product_ID = R.Product_ID
  7  GROUP BY P.Product_ID, P.Product_Name
  8  ORDER BY Total_Reviews DESC;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
----
TOTAL_REVIEWS
-------------
P100001
Gaming Headset
            1

P100003
Mechanical Keyboard
            1

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
----
TOTAL_REVIEWS
-------------

P100002
Wireless Mouse
            1

P100004
Gaming Controller
```

```
----
TOTAL_REVIEWS
-------------
            1

P100005
Smartphone
            0

P100006

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
----
TOTAL_REVIEWS
-------------
Bluetooth Speaker
            0

P100007
Gaming Monitor
            0


PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
----
TOTAL_REVIEWS
-------------
P100008
Power Bank
            0


8 rows selected.
```

Highly Rated Products

```
SQL> SELECT P.Product_ID,
  2         P.Product_Name,
  3         AVG(R.Rating) AS Average_Rating
  4  FROM Product P
  5  JOIN Product_Review R
  6  ON P.Product_ID = R.Product_ID
  7  GROUP BY P.Product_ID, P.Product_Name
  8  HAVING AVG(R.Rating) >= 4
  9  ORDER BY Average_Rating DESC;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
----
AVERAGE_RATING
--------------
P100004
Gaming Controller
             5

P100001
Gaming Headset
             5

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
----
AVERAGE_RATING
--------------

P100002
Wireless Mouse
             4
```

Report 3: Rating Distribution

Number of 5-Star Ratings

```
SQL> SELECT COUNT(*) AS Five_Star_Ratings
  2  FROM Product_Review
  3  WHERE Rating = 5;

FIVE_STAR_RATINGS
-----------------
                2

SQL> SELECT COUNT(*) AS Four_Star_Ratings
  2  FROM Product_Review
  3  WHERE Rating = 4;

FOUR_STAR_RATINGS
-----------------
                1
```

Number of Low-Rated Reviews

```
SQL> SELECT COUNT(*) AS Low_Rated_Reviews
  2  FROM Product_Review
  3  WHERE Rating < 3;

LOW_RATED_REVIEWS
-----------------
                0
```

Complete Rating Summary

```
SQL> SELECT COUNT(*) AS Total_Reviews,
  2         SUM(CASE WHEN Rating = 5 THEN 1 ELSE 0 END) AS Five_Star,
  3         SUM(CASE WHEN Rating = 4 THEN 1 ELSE 0 END) AS Four_Star,
  4         SUM(CASE WHEN Rating < 3 THEN 1 ELSE 0 END) AS Low_Rated
  5  FROM Product_Review;

TOTAL_REVIEWS  FIVE_STAR  FOUR_STAR  LOW_RATED
------------- ---------- ---------- ----------
            4          2          1          0
```

This query summarizes the total number of reviews, 5-star ratings, 4-star ratings and low-rated reviews.

9. Conclusion

The Product Review and Rating Management Module stores customer feedback and product ratings while maintaining relationships with the existing Customer and Product tables. Primary keys, foreign keys and check constraints help maintain data integrity. SQL queries and aggregate functions help retrieve reviews, calculate average ratings, identify highly rated products and analyze customer feedback. These reports support product performance evaluation and help identify products that may require improvement.